

Sadržaj

1.	Uvod	1
1.1.	Motivacija.....	1
1.2.	Cilj projekta	2
2.	Primijenjene metode i biblioteke	3
2.2.	Pristup "Black-Box" modela u strojnom učenju.....	3
2.3.	Primjena "Black-Box" modela	3
2.4.	Biblioteka rainforest u Pythonu za izgradnju modela slučajnih šuma.....	4
3.	Arhitektura sustava	6
3.1.	Objašnjenje ključnih pojmova	6
4.	Skupovi podataka	8
4.1.	Ulazne .csv tablice	8
4.2.	Vizualizacija i filtracija podataka	9
4.3.	Pronalazak stacionarnih intervala.....	16
4.3.1.	Određivanje stacionarnih intervala i određivanje njihove duljine.....	16
4.3.2.	Uklanjanje duplikata i intervala kraćih od zadanog intervala	17
4.3.3.	Traženje presjeka kod intervala	18
4.3.4.	Tablica zona sa stacionarnim intervalima	19
5.	Priprema podataka za treniranje	21
5.1.	Spajanje tablica.....	23
5.1.1.	Spajanje kalorimetara u jedan file	23
5.1.2.	Spajanje Fcu_type_id-jeva u jedan file	24
5.1.3.	Dodavanje zone_id retka	25
6.	Izrada modela	27
6.1.	Povezivanje po svim godinama	27
6.2.	Spajanje zones, kalorimeter, fcu i fcu_hydraulic_model.....	28

6.3.	Izračun termalne snage	29
6.4.	Završna filtracija.....	30
6.5.	Model.....	32
7.	Usporedba izračunatog thermal_power sa thermal_power očitano sa kalorimetra ...	35
7.1.	Priprema podataka	35
7.2.	Sumiranje po svim zonama.....	38
7.3.	Usporedba sa thermal_power na kalorimetru.....	39
8.	Rezultati.....	42
9.	Zaključak	48
9.1.	Model.....	48
9.2.	Usporedba toplinske snage	49
10.	Sažetak.....	51
11.	Literatura	52

1. Uvod

Ekspanzija digitalizacije i sveprisutnost senzorskih tehnologija omogućile su prikupljanje golemih količina podataka iz fizičkih sustava. Ti podaci, analizirani naprednim metodama, pružaju bezprecednu priliku ne samo za razumijevanje već i za precizno predviđanje ponašanja složenih pojava u stvarnom svijetu. Primjena takvih modela predikcije čini kamen temeljac automatizacije procesa, unapređujući njihovu učinkovitost, energetske optimizaciju i autonomiju.

Cilj ovog rada je primijeniti navedene principe na područje upravljanja sustava klimatizacije na 9. katu C zgrade FER-a pomoću podataka prikupljenih u vremenu od 2018. do 2022. godine. Konkretno, rad se fokusira na razvoj i evaluaciju modela strojnog učenja za predviđanje toplinskog ponašanja ventilokonvektora tijekom procesa grijanja i hlađenja. Razvojem pouzdanih prediktivnih modela, ovaj istraživački rad stvara temelj za implementaciju automatiziranih sustava upravljanja koji mogu dinamički optimizirati rad uređaja, smanjiti potrošnju energije i osigurati toplinsku ugodu.

1.1. Motivacija

Motivacija za ovaj istraživački rad proizlazi iz imperativa energetske učinkovitosti i trajne potrebe za smanjenjem operativnih troškova u segmentu upravljanja zgradama. Klimatizacijski sustavi, a posebno ventilokonvektori kao jedan od najraširenijih uređaja, predstavljaju značajan dio ukupne potrošnje energije u stambenim i komercijalnim objektima. Tradicionalni sustavi upravljanja često se oslanjaju na unaprijed postavljene rasporede i reaktivne strategije, umjesto da proaktivno prilagođavaju rad stvarnim, dinamičkim uvjetima okoline i potrebama korisnika. Ova neefikasnost rezultira prekomjernom potrošnjom energije i ubrzanim trošenjem opreme. Stoga, postoji jasna potreba za implementacijom inteligentnih, podatcima vođenih rješenja koja mogu anticipirati toplinsko ponašanje prostora i optimizirati rad ventilokonvektora u stvarnom vremenu.

1.2. Cilj projekta

Cilj ovog rada je primijeniti navedene principe na područje upravljanja zgradnom klimatizacijom. Konkretno, rad se fokusira na razvoj i evaluaciju modela strojnog učenja za predviđanje toplinskog ponašanje ventilokonvektora tijekom procesa grijanja i hlađenja. Razvojem pouzdanih prediktivnih modela, ovaj istraživački rad stvara temelj za implementaciju automatiziranih sustava upravljanja koji mogu dinamički optimizirati rad uređaja, smanjiti potrošnju energije i osigurati toplinsku ugodu.

2. Primijenjene metode i biblioteke

Kao razvojnu okolinu sam koristio JupyterLab koji koristi python kao programski jezik.

Biblioteke u pythonu koje sam koristio su:

Pandas(učitavanje csv fileova i njihovo oblikovanje), numpy(računanje pogrešaka), sklearn.linear_model(linearna regresija), sklearn.model_selection(podjela na test i train skupove podataka), sklearn.metrics(metrike za mjerenje točnosti modela), sklearn.ensemble(RandomForestRegressor za stvaranje black-box modela)

2.2. Pristup "Black-Box" modela u strojnom učenju

U području strojnog učenja, pojam "black-box" modela (modela "crne kutije") referira se na vrlo složene modele čiji su unutarnji mehanizmi i putovi donošenja odluka teški za ljudsku interpretaciju i direktno razumijevanje. Za razliku od tradicionalnih, jednostavnijih modela (npr. linearne regresije) gdje je utjecaj svake ulazne varijable na izlaz jasan i proziran, "black-box" modeli poput dubokih neuronskih mreža ili složenih ensemble metoda donose odluke kroz ogroman broj međusobno povezanih parametara. Iako su ovi modeli često izuzetno precizni i moćni u otkrivanju složenih, nelinearnih obrazaca u podacima, sam proces transformacije ulaza u izlaz ostaje skriven unutar arhitekture modela. Stoga, vjerodostojnost ovih modela ne proizlazi iz naše sposobnosti da pratimo svaki korak njihovog zaključivanja, već iz njihove konzistentno visoke točnosti u predviđanju na neviđenim podacima i statističke validacije njihovih performansi.

2.3. Primjena "Black-Box" modela

Unatoč svojoj neprozivosti, "black-box" pristup ima široku i opravdanu primjenu u rješavanju realnih problema. Njihova primjena je posebno vrijedna u slučajevima gdje je:

1. **Prediktivna točnost ključna:** Kada je glavni cilj postići što je moguće veću točnost predviđanja, a potpuno razumijevanje utjecaja pojedinih varijabli je sekundarno. Primjeri uključuju prepoznavanje slika, preporučivanje proizvoda ili, kao u ovom radu, predviđanje toplinskih procesa gdje je kompleksna fizikalna interakcija teška za modeliranje tradicionalnim metodama.

2. **Složenost problema velika:** Za probleme s golemim brojem ulaznih značajki i složenim, nelinearnim odnosima među njima, "black-box" modeli poput neuronskih mreža su često jedina opcija koja može uhvatiti te složenosti i postići zadovoljavajuće performanse.
3. **Dostupni su veliki skupovi podataka:** Snaga ovih modela u potpunosti se otkriva kada su trenirani na velikim količinama podataka. Podaci pružaju osnovu za vjerodostojnost, nadomještajući nedostatak interpretabilnosti empirijskim dokazima o točnosti.

U kontekstu ovog istraživanja, primjena "black-box" modela opravdana je upravo zbog složenosti dinamike grijanja i hlađenja, koja uključuje brojne čimbenike (vanjska temperatura, izolacija, rad uređaja, itd.) čije međudjelovanje je teško egzaktno opisati. Fokus je stoga na postizanju robusnog i točnog prediktivnog alata za automatizaciju, pri čemu se valjanost modela procjenjuje njegovim rezultatima, a ne mogućnošću detaljnog razlaganja njegovih unutarnjih procesa.

2.4. Biblioteka rainforest u Pythonu za izgradnju modela slučajnih šuma

U ovom radu za izgradnju i evaluaciju modela strojnog učenja temeljenih na metodi slučajnih šuma (Random Forest) korištena je **Rainforest biblioteka**. Rainforest je moćan i moderni Python paket dizajniran za implementaciju ensemble modela koji kombiniraju brojne stabla odluke kako bi postigli visoku prediktivnu točnost i robusnost.

Glavna prednost rainforest biblioteke leži u njenoj optimiranoj implementaciji koja omogućuje efikasnu obradu velikih skupova podataka. Za razliku od klasičnih implementacija, rainforest koristi napredne algoritamske tehnike za ubrzanje procesa treniranja i predikcije, što je osobito korisno pri radu s velikim znanstvenim podacima. Biblioteka nudi široku paletu hiperparametara koji se mogu fino podesiti, uključujući broj stabala u šumi (`n_estimators`), maksimalnu dubinu pojedinačnog stabla (`max_depth`), te minimalni broj uzoraka potrebnih za podjelu čvora (`min_samples_split`).

Osim toga, rainforest biblioteka automatski rukuje i nedostajućim vrijednostima te kategorijskim značajkama, što znatno pojednostavljuje cjelokupni pipeline obrade podataka. Zbog svoje sposobnosti da modelira složene nelinearne odnose i interakcije između značajki, metoda slučajnih šuma primjenjiva je na širok spektar problema, od klasifikacije do

regresije. U kontekstu ovog istraživanja, korištenje ove biblioteke omogućilo je postizanje visoke generalizacijske sposobnosti modela i smanjenje rizika od prenaučenosti.

3. Arhitektura sustava

Podatci s kojima sa bavimo su vezani za 9. kat C zgrade FER-a. Na sjeveru i na jugu zgrade se nalaze po jedan kalorimetar. Sjeverni i južni dio zgrade su podijeljeni u zone (u stvarnom svijetu prostorije). U svakoj zoni se nalazi FCU uređaj koji očitava return temperaturu dok se na kalorimetru očitava supply temperatura. Razlikom te dvije vrijednosti dobijemo delta T koji se koristi za računanje potrošnje termalne energije.

Opis sustava i podataka

Istraživanje u ovom radu temelji se na podacima prikupljenim s 9. kata C zgrade FER-a. Prostor kata podijeljen je na **sjevernu** i **južnu zonu**, koje odgovaraju skupinama prostorija s različitom orijentacijom i toplinskim karakteristikama. Svaka zona opskrbljuje se toplinskom energijom iz zasebnog razvodnog kruga, a njihova potrošnja se mjeri nezavisno.

Ključni uređaji za prikupljanje podataka su:

- **Kalorimetri:** Na ulazima u sjeverni i južni razvodni krug postavljen je po jedan kalorimetar. Ovi uređaji kontinuirano mjere **temperaturu opskrbe vode (Supply Temperature, Ts)** koja ulazi u sustav zone.
- **FCU uređaji (Ventilokonvektori):** Unutar svake prostorije (zone) postavljen je FCU uređaj. Svaki od njih mjeri **temperaturu povratne vode (Return Temperature, Tr)** koja izlazi iz tog pojedinačnog uređaja nakon što je izmijenila toplinu sa zrakom u prostoriji.

Delta T (ΔT) se za svaki FCU uređaj računa kao razlika izmjerenih temperatura: $\Delta T = T_s - T_r$.

Ova razlika temperature između opskrbe i povratne vode direktno je proporcionalna količini toplinske energije predane u prostoriju. Veća razlika (ΔT) indicira veći prijenos topline i samim time veću potrošnju energije. Stoga je ΔT ključna veličina za analizu učinkovitosti i potrošnje, a korištenje podataka s više FCU uređaja unutar zone omogućuje detaljniju analizu toplinskog ponašanje cijelog sustava.

3.1. Objašnjenje ključnih pojmova

FCU (Fan Coil Unit)

FCU (engl. Fan Coil Unit), na hrvatskom se često naziva ventilokonvektor, je krajnji uređaj u sustavu grijanja i hlađenja koji se direktno nalazi u prostoriji koju hladi/grije. Njegov rad

temelji se na prolasku zraka iz prostorije (koji potiskuje ventilator) kroz izmjenjivač topline – serpentinu kroz koju cirkulira topla ili hladna voda. Zrak se zagrijava ili hladi u kontaktu sa serpentinom, te se potom vrati u prostoriju. Upravo ovaj uređaj mjeri temperaturu vode nakon što je izmijenila toplinu sa zrakom, odnosno povratnu temperaturu (T_r).

Kalorimetar

Kalorimetar je mjerni uređaj postavljen na razvodnom cjevovodu primarne vode, čija je svrha točno mjerenje potrošnje toplinske (ili rashladne) energije u dijelu sustava koji opslužuje (npr. sjeverna ili južna zona). On ne mjeri samo temperaturu, već i protok vode kroz sustav. Energija se računa na temelju razlike temperature između ulazne (opskrbe) i izlazne (povratne) vode te volumena vode koja je prošla kroz sustav. U kontekstu ovog rada, koristi se podatak o temperaturi opskrbe vode (T_s) koju on mjeri.

Ventilokonvektor

Ventilokonvektor je hrvatski naziv za FCU uređaj.

4. Skupovi podataka

4.1. Ulazne .csv tablice

calorimeter_23_year_2018-

- 23 - broj kalorimetra(u ovom slučaju sjeverni dio 11. kata)
- 2018 - godina u kojoj su prikupljeni podatci iz tablice

Tablica sadrži:

- 'id' – identifikacijsku broj tog očitavanja
- 'calorimeter_id'- identifikacijski broj kalorimetra
- 'batch_timestamp'—prikaz vremena
- 'timestamp',--prikaz vremena
- 'supply_temperature',--temperatura na ulazu sustava
- 'return_temperature',--temperatura na izlazu sustava
- 'temperature_difference',--razlika između supply i return
- 'mass_volume_flow', -- volumni protok vode kroz kalorimetar
- 'thermal_power', ---termalna snaga potrošena u tom trenutku
- 'thermal_energy_heating',---termalna energija potrošena na grijanje
- 'thermal_energy_cooling', --termalna energija potrošena na hlađenje
- 'error'—Nan ako nema greške

Na raspolaganju smo također imali tablice sa istim parametrima samo za godine 2019, 2020, 2021, 2022, te za kalorimetar_id 24(južni dio kata).

zones_23_year_2018

- 'id', identifikacijsku broj tog očitavanja
- 'zone_id', --identifikacijski broj zone
- 'batch_timestamp', prikaz vremena
- 'timestamp', prikaz vremena
- 'zone_temperature',--- temperatura u zoni

- 'zone_fan_speed',--- brzina vrtnje ventilokonvektora
- 'smart_control',--pametni način rada ventilokonvektora
- 'local_switch',--manulani način rada ventilokonvektora
- 'temperature_setpoint'—temperatura na koju je namješten ventilokonvektor

fcus_23_year_2018

- fcu_id—identifikacijski broj fcu-a
- return_medium_temperature—temperatura očitana na izlazu fcu-a

fcu_hydraulic_model.csv

- fcu_id- Identifikacijski broj fcu uređaja
- timestamp- trenutak mjerenja podatka
- flow_share- prikaz udjela supply_temperature-a koji iz kalorimetra dolazi do fcu uređaja

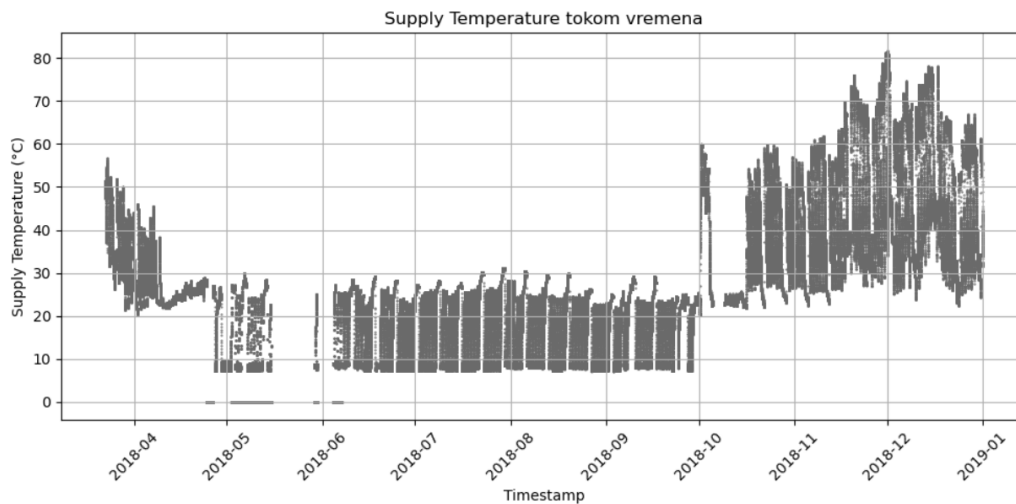
fcus.csv- sadrži podatke kao i fcus_23_year_2018 samo za sve kalorimetre i za sve godine uz dodatni parametar zone_id.

4.2. Vizualizacija i filtracija podataka

Najprije je bilo potrebno vizualizirati podatke kako bi odredili njihovo ponašanje. To samo napravili pomoću grafova u pythonu. Također smo na svakom parametru maknuli rubne vrijednosti pomoću 3-sigma pravila, te slučajeve koji nas ne zanimaju(npr. Kada su određene vrijednosti 0).

```
filtered_data = df[df['supply_temperature'] != 0]
mu = df['supply_temperature'].mean()
sigma = df['supply_temperature'].std()

filtered_df = df[(df['supply_temperature'] >= mu - 3*sigma) &
                 (df['supply_temperature'] <= mu + 3*sigma)]
```



Sl. 4.1 Supply temperature za kalorimetar 24 u 2018.godini

Vizualno iz grafa možemo odrediti da je granica između hlađenja i grijanja oko 25 stupnjeva.

```
df['interval_d'] = df['timestamp'].dt.to_period('D')
```

```
grupe_intervala = df.groupby('interval_d')
```

```
rezultat = []
```

```
for interval, group in grupe_intervala:
```

```
    ukupno = len(group)
```

```
    ispod_25 = len(group[group['supply_temperature'] < 25])
```

```
    iznad_25 = ukupno - ispod_25
```

```
    vecina = "Ispod 25°C" if ispod_25 > iznad_25 else "Iznad 25°C"
```

```
    rezultat.append({
```

```
        'interval': interval,
```

```
        'ukupno_podataka': ukupno,
```

```
        'ispod_25': ispod_25,
```

```
        'iznad_25': iznad_25,
```

```

        'vecina': vecina

    })

df_intervals = pd.DataFrame(rezultat)

start_date = pd.Period("2018-04-15", freq='D')
end_date = pd.Period("2018-05-01", freq='D')

konacni = df_intervals[(df_intervals['interval'] > start_date) &
(df_intervals['interval'] < end_date)]

```

	interval	ukupno_podataka	ispod_25	iznad_25	vecina
25	2018-04-16	1431	926	505	Ispod 25°C
26	2018-04-17	1438	618	820	Iznad 25°C
27	2018-04-18	1433	0	1433	Iznad 25°C
28	2018-04-19	1439	361	1078	Iznad 25°C
29	2018-04-20	1439	1	1438	Iznad 25°C
30	2018-04-21	1439	0	1439	Iznad 25°C
31	2018-04-22	1440	0	1440	Iznad 25°C
32	2018-04-23	1437	20	1417	Iznad 25°C
33	2018-04-24	1413	672	741	Iznad 25°C
34	2018-04-25	1427	1427	0	Ispod 25°C
35	2018-04-26	1440	672	768	Iznad 25°C
36	2018-04-27	1439	1063	376	Ispod 25°C
37	2018-04-28	1438	752	686	Ispod 25°C
38	2018-04-29	1439	1357	82	Ispod 25°C
39	2018-04-30	1440	1440	0	Ispod 25°C

Sl. 4.2 Tablica prikaza perioda hlađenja i grijanja

Zatim smo prikazali još jedan interval u kojem dolazi do promjene grijanja i hlađenja.

```

start_date = pd.Period("2018-10-05", freq='D')
end_date = pd.Period("2018-10-20", freq='D')

```

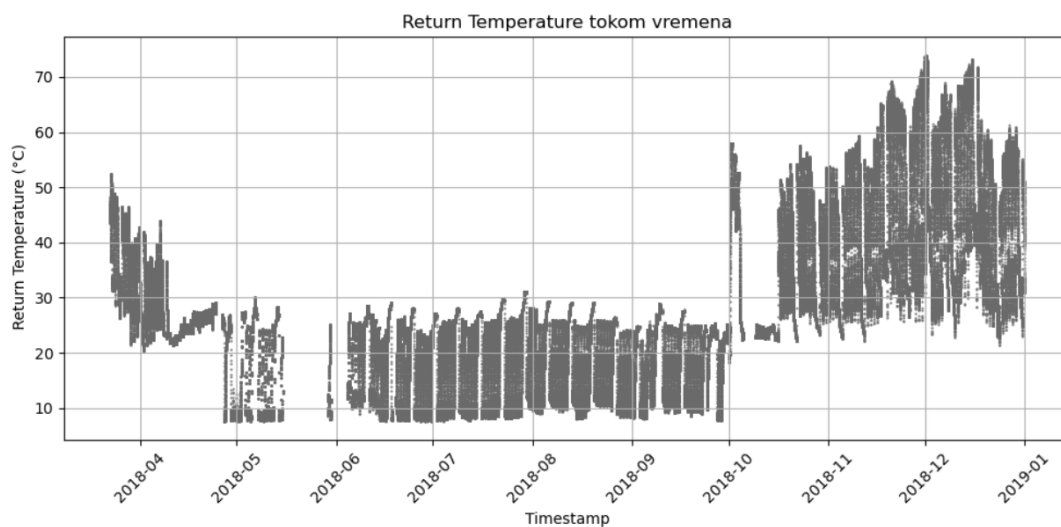
```
konacni = df_intervals[(df_intervals['interval'] > start_date) &
(df_intervals['interval'] < end_date)]
```

	interval	ukupno_podataka	ispod_25	iznad_25	vecina
181	2018-10-09	928	896	32	Ispod 25°C
182	2018-10-10	1439	1437	2	Ispod 25°C
183	2018-10-11	1440	1436	4	Ispod 25°C
184	2018-10-12	1440	1435	5	Ispod 25°C
185	2018-10-13	1440	1426	14	Ispod 25°C
186	2018-10-14	1440	1231	209	Ispod 25°C
187	2018-10-15	1440	1437	3	Ispod 25°C
188	2018-10-16	1440	524	916	Iznad 25°C
189	2018-10-17	1439	0	1439	Iznad 25°C
190	2018-10-18	1440	0	1440	Iznad 25°C
191	2018-10-19	1440	0	1440	Iznad 25°C

Sl. 4.3 tablica prikaza perioda hlađenja i grijanja

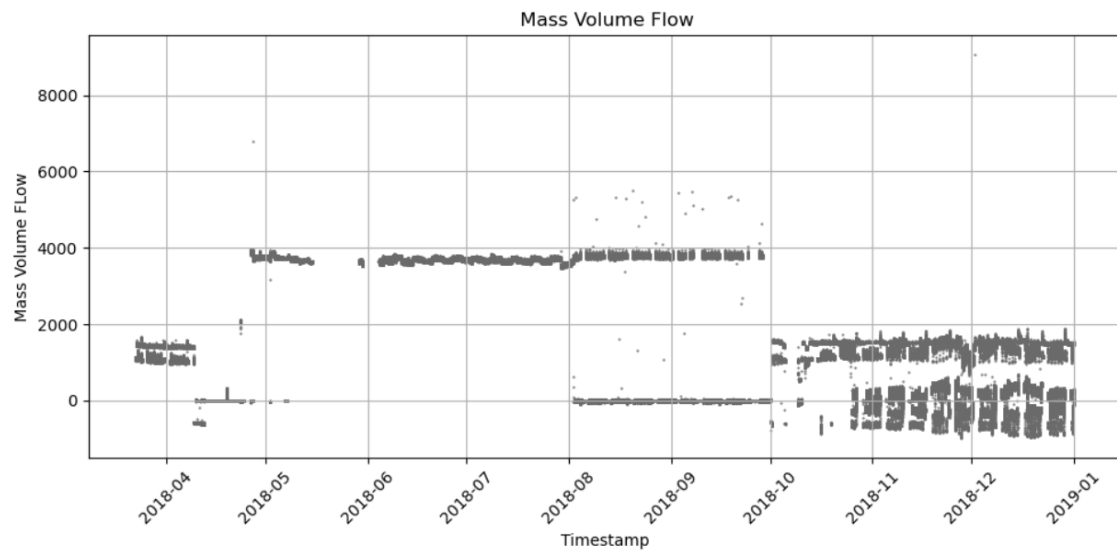
Iz sljedećih podataka možemo zaključiti da je 2018-10-16 završio period hlađenja, a krenuo period grijanja.

Return Temperatre



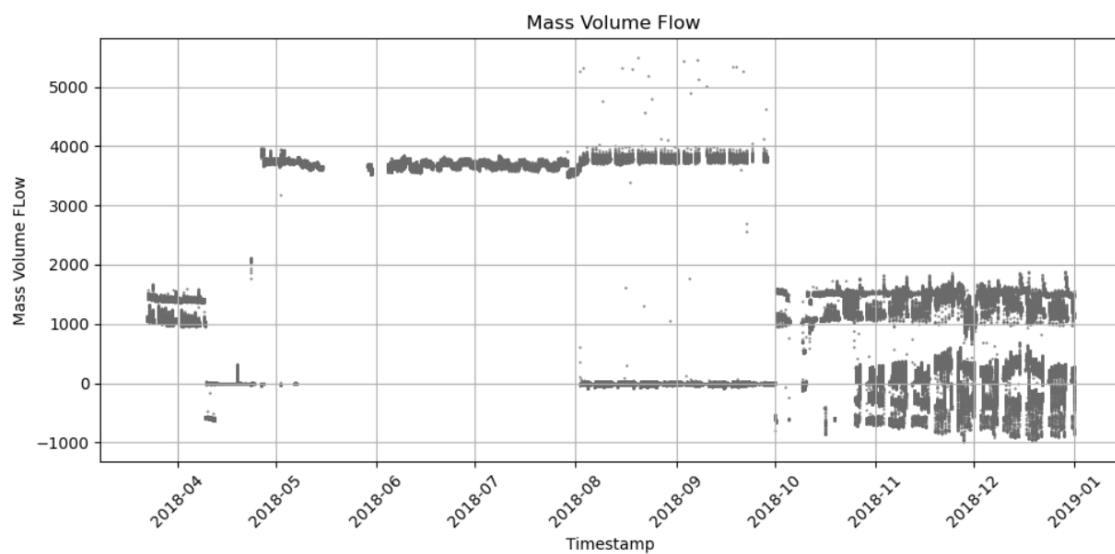
Sl. 4.4 Return Temperature za kalorimetar 24 u 2018.godini

mass_volume_flow



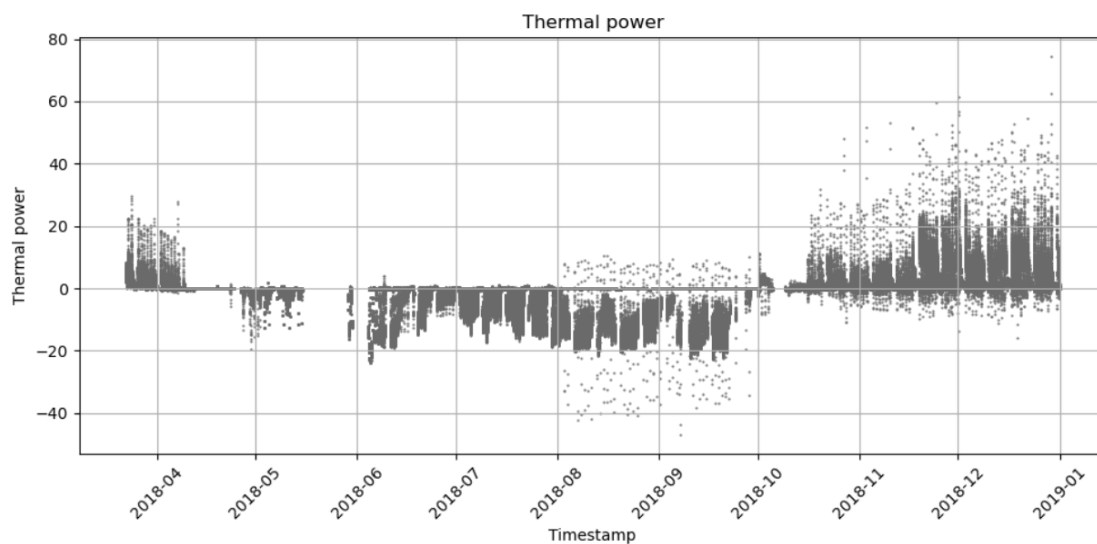
Sl. 4.5 mass volume flow za kalorimetar 24 u 2018.godini

Zbog velikog odstupanja u određenim trenucima je potrebno prikazati graf od mass_volume_flow nakon primjene 3-sigma.



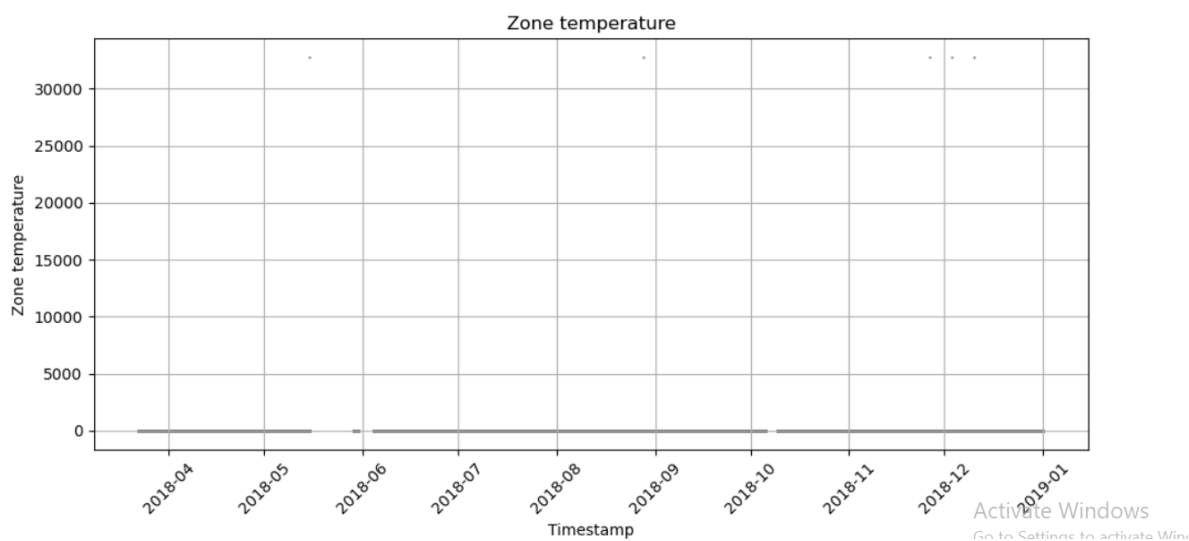
Sl. 4.6 mass volume flow za kalorimetar 24 u 2018.godini nakon 3-sigma

thermal_power



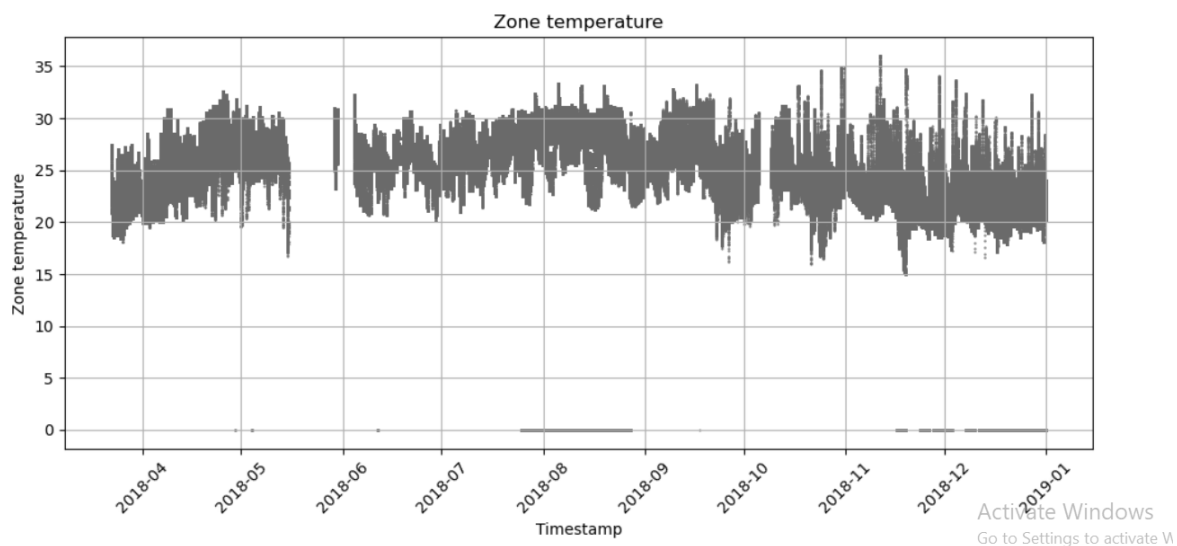
Sl. 4.7 thermal power za kalorimetar 24 u 2018.godini

Zone_temperature



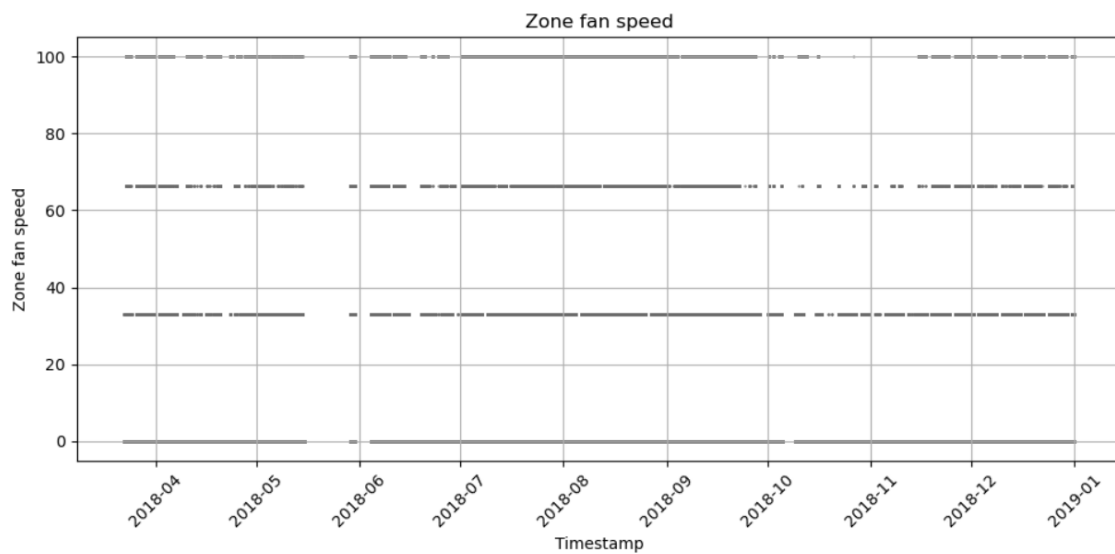
Sl. 4.8 zone temperature za kalorimetar 24 u 2018.godini

Zbog velikog odstupanja u određenim trenucima je potrebno prikazati graf od mass_volume_flow nakon primjene 3-sigma.



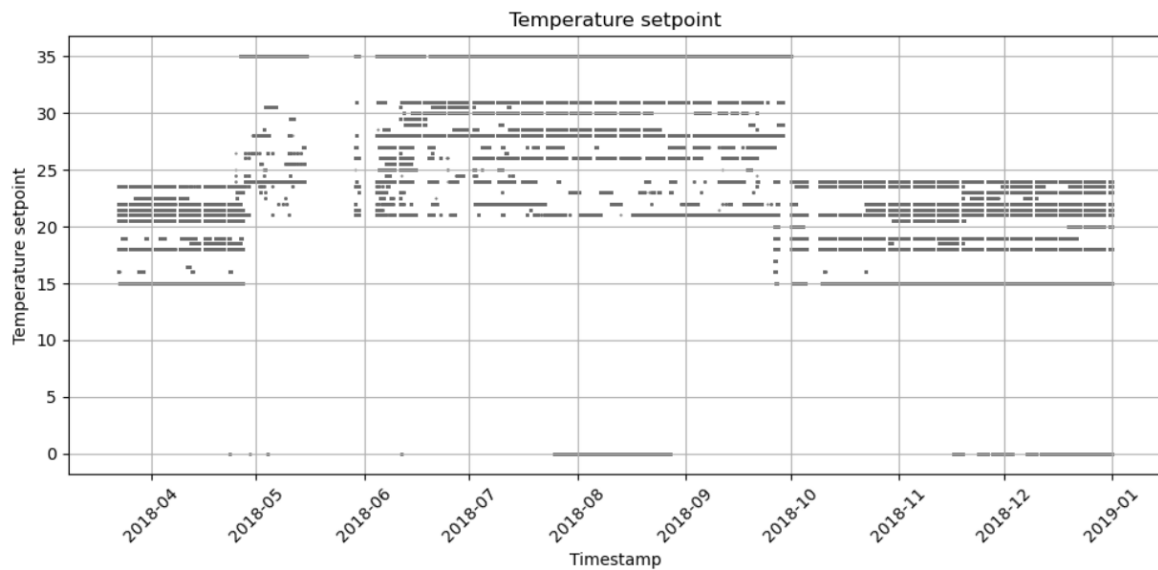
Sl. 4.9 zone temperature za kalorimetar 24 u 2018.godini nakon 3-sigma

zone_fan_speed



Sl. 4.10 zone fan speed za kalorimetar 24 u 2018.godini

temperature_setpoint



Sl. 4.11 temperature setpoint za kalorimetar 24 u 2018.godini

Maknuli smo podatke u kojima je temperature setpoint 0 jer promatramo podatke kada je ventilokonvektor aktivan.

```
filtered_data = df1[df1['temperature_setpoint'] != 0]

df1.to_csv('../../../../results/2018/24/zones_24_year_2018filtriranje.csv',
index=False)
```

4.3. Pronalazak stacionarnih intervala

4.3.1. Određivanje stacionarnih intervala i određivanje njihove duljine

Najprije je bilo potrebno odrediti intervale u kojima je brzina vrtnje ventilokonvektora (zone_fan_speed) nije mijenjala.

```
file_path1="../../../../results/2018/24/zones_24_year_2018filtriranje.csv"
data1=pd.read_csv(file_path1, index_col=[0])

import pandas as pd
```

Određivanje intervala i određivanje njegove duljine

```
data1['timestamp'] = pd.to_datetime(data1['timestamp']).dt.round('min')

all_intervals = []
```

```

for zone_id in range(195, 208):

    zone_data = data1[data1['zone_id'] == zone_id]

    zone_data_sorted = zone_data.sort_values(by='timestamp')

    zone_data_sorted['change'] = zone_data_sorted['zone_fan_speed'] !=
zone_data_sorted['zone_fan_speed'].shift()

    zone_data_sorted['group'] = zone_data_sorted['change'].cumsum()

    intervals = zone_data_sorted.groupby('group').agg(

        start_time=('timestamp', 'first'),

        end_time=('timestamp', 'last'),

        speed=('zone_fan_speed', 'first')

    ).reset_index(drop=True)

    intervals['duration']=(intervals['end_time']-
intervals['start_time']).dt.total_seconds() / 3600

    intervals['zone_id'] = zone_id

    all_intervals.append(intervals)

all_intervals_df = pd.concat(all_intervals, ignore_index=True)

```

4.3.2. Uklanjanje duplikata i intervala kraćih od zadanog intervala

U ovom koraku uklanjamo duplikate intervala(ako su nastupili) , te filtriramo intervale tako da biramo one koji su dulji od zadanog intervala(imat ćemo varijacije sa 15 minuta, 30 minuta, 1 sat).

```

all_intervals_df = pd.concat(all_intervals, ignore_index=True)

all_intervals_df['start_time']=
pd.to_datetime(all_intervals_df['start_time'])

```

```

all_intervals_df['end_time']=
pd.to_datetime(all_intervals_df['end_time'])

all_intervals_df['duration']      =      all_intervals_df['end_time']      -
all_intervals_df['start_time']

filtered_intervals_df = all_intervals_df[all_intervals_df['duration'] >=
pd.Timedelta(minutes=15)]

unique_intervals      =      filtered_intervals_df[['start_time',      'end_time',
'zone_id', 'speed']].drop_duplicates()

```

4.3.3. Traženje presjeka kod intervala

Tražimo intervale u kojima su sve zone u stacionarnom stanju.

```

intersections = []

for _, interval in unique_intervals.iterrows():
    start = interval['start_time']
    end = interval['end_time']
    current_zone = interval['zone_id']
    speed= interval['speed']

    overlapping_zones = filtered_intervals_df[
        (filtered_intervals_df['zone_id'] != current_zone) &
        (filtered_intervals_df['start_time'] <= end) &
        (filtered_intervals_df['end_time'] >= start)
    ]

    covered_zones = set(overlapping_zones['zone_id'])
    required_zones = set(range(197, 208)) - {current_zone}

    if required_zones.issubset(covered_zones):

```

```

        intersection_start = max(overlapping_zones['start_time'].min(),
start)

        intersection_end = min(overlapping_zones['end_time'].max(), end)

        intersection_duration = intersection_end - intersection_start

    intersections.append({

        'zone_id': current_zone,

        'intersection_start': intersection_start,

        'intersection_end': intersection_end,

        'intersection_duration': intersection_duration,

        'speed': speed

    })

intersections_df = pd.DataFrame(intersections)

only_end = intersections_df[["intersection_end", "speed"]]

only_end.to_csv('../../../../results/2018/24/15min/calorimeter_24_year_20
18presjeci.csv', index=False)

```

4.3.4. Tablica zona sa stacionarnim intervalima

Zatim koristeći `intersection_end` iz `calorimeter_24_year_2018presjeci.csv` iz tablice `zones_24_year_2018filtriranje.csv` mogu izdvojiti podatke vezane za zone u trenucima kada se nalaza u stabilnom stanju

```

file_path=
"../../../../../results/2018/24/zones_24_year_2018filtriranje.csv"

file_path1=
"../../../../../results/2018/24/15min/calorimeter_24_year_2018presjeci.csv"

data = pd.read_csv(file_path, index_col=[0])

data1=pd.read_csv(file_path1, index_col=[0])

```

```

data1 = data1.reset_index()

data1['intersection_end']=
pd.to_datetime(data1['intersection_end']).dt.round('min')

data['timestamp'] = pd.to_datetime(data['timestamp']).dt.round('min')

merged      =      pd.merge(data1,      data,      left_on='intersection_end',
right_on='timestamp', how='inner')

merged.to_csv(
'../../../../results/2018/24/15min/zone_stac_stanja_24_2018.csv',
index=False)

```

U ovom odjeljku je prikazan postupak generiranja tablice zone_stac_stanja_24_2018.csv koja prikazuje tablice zona za stacionarna stanja.

Takav proces je bilo potrebno ponoviti i za ostale godine(2019, 2020, 2021, 2022), intervale (30 minuta, 1 sat) , kalorimetar (23).

5. Priprema podataka za treniranje

Najprije smo iz tablice kalorimetar odrediti koji su calorimeter_id za kalorimetre na sjevernom i južnom dijelu 11.kata. Zatim smo odredili zone koje pripadaju tom području i njihove zone_id.

```
import pandas as pd
import matplotlib.pyplot as plt

%matplotlib inline

file_path_zone="data/zone.csv"
file_path_calorim="data/calorimeter.csv"
file_path_fcu="data/fcu.csv"

data_zone=pd.read_csv(file_path_zone)
data_calorim=pd.read_csv(file_path_calorim)
data_fcu=pd.read_csv(file_path_fcu)

calorillsjev=data_calorim.loc[data_calorim['name']=='C11N']
print(f"measurement_group_id      za      11.      kat      sjever      je
{calorillsjev['calorimeter_id'].tolist()}")
measurement_group_id za 11. kat sjever je [23]

calorillsjev=data_calorim.loc[data_calorim['name']=='C11S']
print(f"measurement_group_id      za      11.      kat      jug      je
{calorillsjev['calorimeter_id'].tolist()}")
measurement_group_id za 11. kat jug je [24]

filtrirane_zone24=data_zone.loc[data_zone['measurement_group_id']==24]
print(f"zone za 11. kat jug su {filtrirane_zone24['zone_id'].tolist()}")

filtrirane_zone24=data_zone.loc[data_zone['measurement_group_id']==23]
print(f"zone      za      11.      kat      sjever      su
{filtrirane_zone24['zone_id'].tolist()}")
```

Rezultati su:

Kalorimetar ID	Zone ID
23	193, 194, 208, 209, 210, 211, 212
24	195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207

Sl. 5.1 poveznica kalorimetar ID i zone ID

Zatim je bilo potrebno izdvojiti koji zone_id pripada kojem fcu_type_id.

Sljedećim kodom smo došli do potrebnih informacija za kalorimetar 23.

```

zone_ids = [193, 194, 208, 209, 210, 211, 212]

filtered_fcu_north = data_fcu.loc[data_fcu['zone_id'].isin(zone_ids)]

fcu_type_id2 = filtered_fcu_north[filtered_fcu_north['fcu_type_id'] == 2]
fcu_id2_list = fcu_type_id2['fcu_id'].unique().tolist()

fcu_type_id2_zone = filtered_fcu_north[filtered_fcu_north['fcu_type_id']
== 2]
fcu_id2_list_zone = fcu_type_id2_zone['zone_id'].unique().tolist()

fcu_type_id1 = filtered_fcu_north[filtered_fcu_north['fcu_type_id'] == 1]
fcu_id1_list = fcu_type_id1['fcu_id'].unique().tolist()

fcu_type_id1_zone = filtered_fcu_north[filtered_fcu_north['fcu_type_id']
== 1]
fcu_id1_list_zone = fcu_type_id1_zone['zone_id'].unique().tolist()

print("FCU IDs with type 1:", fcu_id1_list)
print("FCU IDs with type 2:", fcu_id2_list)

```

```
print("Zone IDs with type 1:", fcu_id1_list_zone)
```

```
print("Zone IDs with type 2:", fcu_id2_list_zone)
```

Isti proces je bilo potrebno ponoviti za kalorimetar 24. Rezultati su sljedeći:

Kalorimetar ID	Fcu_type_ID	Zone_ID
23	1	X
23	2	193, 194, 208, 209, 210, 211, 212
24	1	195, 196, 197, 198, 199, 201, 202, 204, 205, 206, 207
24	2	200, 203

Sl. 5.2 poveznica fcu type ID sa zone ID

5.1. Spajanje tablica

5.1.1. Spajanje kalorimetara u jedan file

Kako bi na jednom mjesto imali podatke za određeni kalorimetar filtrirane podatke smo spojili u jedan file „calorimetar24.csv“. Isti postupak smo ponovili i za kalorimetar 23.

```
import pandas as pd
```

```
from pathlib import Path
```

```
floor_ids = [24]
```

```
years = [2018, 2019, 2020, 2021]
```

```
all_data = []
```

```
for year in years:
```

```
    for floor_id in floor_ids:
```

```

        path = f"results/{year}/{floor_id}/calorimeter_{floor_id}_year_{year}filtriranje.csv"

    if Path(path).exists():
        df = pd.read_csv(path)
        all_data.append(df)
    else:
        print(f"File not found: {path}")

if all_data:
    combined_data = pd.concat(all_data, ignore_index=True)
    combined_data.to_csv("results/calorimeter_filtered24.csv",
index=False)

```

5.1.2. Spajanje Fcu_type_id-jeva u jedan file

```

import pandas as pd
from pathlib import Path

floor_ids = [23]
years = [2018, 2019, 2020, 2021, 2022]
fcu_ids = [307, 308, 309, 310, 299, 300, 301, 302, 303, 304, 305, 306]
all_data = []

for year in years:
    for floor_id in floor_ids:
        path = f"data/{year}/{floor_id}/fcus_{floor_id}_year_{year}.csv"
        if Path(path).exists():
            df = pd.read_csv(path)
            filtered_df = df[df['fcu_id'].isin(fcu_ids)]

```

```

        all_data.append(filtered_df)
    else:
        print(f"File not found: {path}")

if all_data:
    combined_data = pd.concat(all_data, ignore_index=True)
)

combined_data.to_csv("results/23/fcu_type_id2.csv", index=False)

```

5.1.3. Dodavanje zone_id retka

Zatim je u tim tablicama bilo potrebno nadodati redak zone_id.

```

import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

file_path_fcu="data/fcu.csv"
data_fcu=pd.read_csv(file_path_fcu)

import pandas as pd

df = pd.read_csv("results/24/
fcu_type_id2.csv")

mapping = pd.read_csv("results/zone_find.csv")

fcu_to_zone = {}

for _, row in mapping.iterrows():
    zone = row["zone_id"]
    if isinstance(row["fcu_id"], str):
        fcu_list = row["fcu_id"].strip("[]").replace(", ", " ").split()
        fcu_list = [int(x) for x in fcu_list]

```

```
else:

    fcu_list = [row["fcu_id"]]

    for fcu in fcu_list:

        fcu_to_zone[fcu] = zone

df["zone_id"] = df["fcu_id"].map(fcu_to_zone)

df.to_csv("results/24/fcu_type_id2_with_zone.csv", index=False)
```

Isti proces smo ponovili za kalorimetar 23 i fcu_type_id 2.

6. Izrada modela

Bilo je potrebno napraviti modele za svaki kalorimetar, svaki tip FCU uređaja(1 ili 2), brzinu vrtnje i interval. Kao inpute modela uzeli smo supply_temperature i zone_temperature, dok je output bio izračunata termalna snaga.

6.1. Povezivanje po svim godinama

U sljedećem kodu ćemo iterirati po svim godinama za sve zone koje imaju fcu_id1 i brzinu 1 i interval od 15 minuta, te spojiti sve podatke u jednu .csv datoteku.

```
import pandas as pd

from pathlib import Path

floor_id = 24
years = [2018, 2019, 2020, 2021, 2022]
zone_ids = [195, 196, 197, 198, 199, 201, 202, 204, 205, 206, 207]
interval= "15min"
all_data = []

for year in years:
    path=
    f"../../../../../results/{year}/{floor_id}/{interval}/zone_stac_stanja_{f
loor_id}_{year}.csv"

    if Path(path).exists():
        df = pd.read_csv(path)

        filtered_df=df[(df['zone_id'].isin(zone_ids)) &
(df['zone_fan_speed'] == 33)]

        all_data.append(filtered_df)
```

```

else:

    print(f"File not found: {path}")

combined_data.to_csv("../../../../../results/fcu_type_id1/24/speed1/15min/
zones.csv", index=False)

```

6.2. Spajanje zones, calorimeter, fcu i fcu_hydraulic_model

Zatim spajamo zones tablicu sa calorimeter tablicama kako bi dobili supply_temp u tome trenutku. Zatim spajamo sa fcu tablicama za određeni fcu_type_id (u ovom slučaju 1) kako bi dobili return_medium_temperature za tu zonu u tom određenom trenutku.

```

import pandas as pd

calorim= pd.read_csv("../../../../../results/calorimeter_filtered24.csv")

zones=
pd.read_csv("../../../../../results/fcu_type_id1/24/speed1/15min/zones.csv")

fcu_files=
pd.read_csv("../../../../../results/24/fcu_type_id1_with_zone.csv")

koef= pd.read_csv("../../../../../data/fcu_hydraulic_model.csv")

calorim['timestamp']=
pd.to_datetime(calorim['timestamp']).dt.round('min')

zones['timestamp'] = pd.to_datetime(zones['timestamp']).dt.round('min')

koef['timestamp'] = pd.to_datetime(koef['timestamp']).dt.round('min')

fcu_files['timestamp']=
pd.to_datetime(fcu_files['timestamp']).dt.round('min')

calorim = calorim.sort_values('timestamp')

zones = zones.sort_values('timestamp')

merged = pd.merge(zones, calorim, on='timestamp', how='left')

```

```

final_merged1 = pd.merge(merged, fcu_files, on=['timestamp', 'zone_id'],
how='left')

koef = koef.rename(columns={'timestamp': 'timestamp_koef'})

final_merged = pd.merge(final_merged1, koef, on='fcu_id', how='left')

final_merged = final_merged.sort_values('timestamp')

final_merged.to_csv("../.../results/usporedbaThermal/24/speed1/15
min/merged_data.csv", index=False)

final_merged.to_csv("../.../results/fcu_type_id1/24/speed1/15min/
merged_data.csv", index=False)

```

6.3. Izračun termalne snage

Sada imamo sve podatke koji su nam potrebni za računanje istrošene termalne snage koji računamo po formuli:

$$Q = \text{mass_volume_flow} * \text{cp} * \Delta T * (\text{flow_share} / 100)$$

Q = toplinska snaga (kW)

mass_volume_flow = maseni protok (kg/s)

cp = specifični toplinski kapacitet vode (4184 J/(kg·K))

ΔT = temperaturna razlika između dovodne i povratne temperature (K ili °C)

Flow_share = postotak od supply_temp koji iz kalorimetra dolazi do ventilokonvektora.

```

final_merged=
pd.read_csv("../.../results/fcu_type_id1/24/speed1/15min/merged_d
ata.csv")

```

```

CP_WATER = 4184 # J/(kg·K)

```

```

final_merged['mass_flow_kg_s'] = (final_merged['mass_volume_flow'] *
final_merged['flow_share'] / 100 ) / 3600

```



```

# ΔT

final_merged['delta_T'] = (final_merged['supply_temperature'] *
final_merged['flow_share']) - final_merged['return_medium_temperature']


# Toplinska energija u kW

final_merged['calculated_thermal_power'] = (
    final_merged['mass_flow_kg_s'] * CP_WATER * final_merged['delta_T'] /
    1000
)


final_merged.to_csv("../.../.../.../results/fcu_type_id1/24/speed1/15min/
calc_thermal.csv", index=False)

```

6.4. Završna filtracija

Zatim je bilo potrebno iz izračunatih podataka maknuti one koji znatno odstupaju. To smo postigli pomoću linearne regresije i 3-sigma pravila. Kao faktore sam uzeo korelaciju između supply_temperature i izračunate toplinske snage. Također smo izbacili podatke u kojima je mass_volume_flow manji od -100 ili veći od 100 budući da tada nema protoka kroz medij što je indikator da sustav nije aktivan. Zbog velikog odstupanja kada je supply temperature oko 0 sam maknuo podatke kojima je supply_temp manji od 10.

```

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression

import numpy as np


zone_ids = [195, 196, 197, 198, 199, 201, 202, 204, 205, 206, 207]


filtered_all = []


for zid in zone_ids:
    df_zone = df[(df['zone_id'] == zid) & (df['supply_temperature'] >= 20)
].reset_index(drop=True)

```

```

X = df_zone[['supply_temperature']].values
y = df_zone['calculated_thermal_power'].values

model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)

residuals = y - y_pred
sigma = np.std(residuals)

mask = np.abs(residuals) <= 3 * sigma
df_filtered = df_zone[mask].copy()
df_filtered['y_pred'] = y_pred[mask]
filtered_all.append(df_filtered)

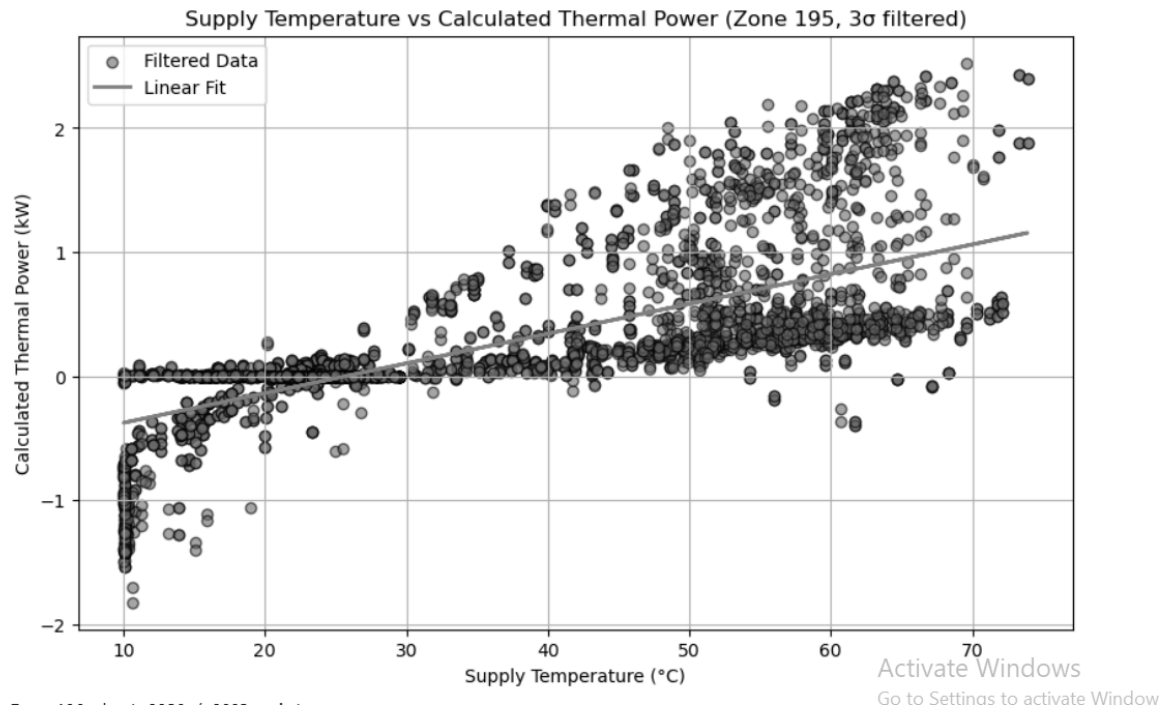
plt.show()

final_filtered1 = pd.concat(filtered_all, ignore_index=True)
final_filtered = final_filtered1[(final_filtered1['mass_volume_flow'] < -
100) | (final_filtered1['mass_volume_flow'] > 100)]

final_filtered.to_csv("../../../../../results/fcu_type_id1/24/speed1/15mi
n/merged_filtered.csv", index=False)

```

Zone 195: kept 3875 / 3900 points



Sl. 6.1 odnos supply_temp i toplinske energije nakon filtracije

6.5. Model

Sada imamo sve podatke potrebne za strojno učenje. Kao ulazne parametre biram supply_temperature i zone_temperature dok za izlazni biram calculated_thermal_power. Kako bi dobili najbolji uvid o točnosti podataka koristili smo K-fold podjelu podataka na test i train. Za izradu grafa odnosa realne i predviđene snage sam podijelio skup podataka na 80% train i 20% test. Podatci svih metrika točnosti za sve primjere su prikazani u odlomku Rezultati.

```
import pandas as pd

from sklearn.model_selection import train_test_split, cross_val_score,
KFold

from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_squared_error, r2_score

import matplotlib.pyplot as plt

import numpy as np

file_path=
"../../../../../results/usporedbaThermal/24/speed1/15min/merged_filtered.
csv"
```

```

data = pd.read_csv(file_path)

X = data[["supply_temperature", "zone_temperature"]]
y = data["calculated_thermal_power"]

data = pd.concat([X, y], axis=1).dropna()

data_sampled = data.sample(frac=0.5, random_state=42)

X = data_sampled[["supply_temperature", "zone_temperature"]]
y = data_sampled["calculated_thermal_power"]

model = RandomForestRegressor(n_estimators=20, max_depth=10,
                              random_state=42)

import seaborn as sns

from sklearn.model_selection import KFold, cross_val_score,
cross_val_predict

from sklearn.metrics import (mean_squared_error, mean_absolute_error,
                             r2_score, mean_absolute_percentage_error,
                             explained_variance_score, max_error)

kfold = KFold(n_splits=5, shuffle=True, random_state=42) # 5-fold CV

cv_scores = cross_val_score(model, X, y, cv=kfold, scoring='r2')

mse_scores = -cross_val_score(model, X, y, cv=kfold,
                              scoring='neg_mean_squared_error')

y_pred_cv = cross_val_predict(model, X, y, cv=kfold)

mae = mean_absolute_error(y, y_pred_cv)

rmse = np.sqrt(mean_squared_error(y, y_pred_cv))

if (y != 0).all():

```

```

mape = mean_absolute_percentage_error(y, y_pred_cv) * 100
else:
    mape = np.nan

explained_variance = explained_variance_score(y, y_pred_cv)
max_err = max_error(y, y_pred_cv)

mae_scores = -cross_val_score(model, X, y, cv=kfold,
                              scoring='neg_mean_absolute_error')
rmse_scores = np.sqrt(mse_scores)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
plt.scatter(y_test, y_pred, alpha=0.6)
plt.xlabel("Actual calculated thermal power")
plt.ylabel("Predicted calculated thermal power")
plt.title("Actual vs Predicted Thermal Power")
plt.grid(True)
plt.show()

```

7. Usporedba izračunatog thermal_power sa thermal_power očitano sa kalorimetra

Ponavljamo isti postupak kao i pri stvaranju modela samo što sada ne trebamo zone koje pripadaju određenom fcu_id_type nego sve zone na tom kalorimetru.

7.1. Priprema podataka

```
import pandas as pd
from pathlib import Path

floor_id = 24
years = [2018, 2019, 2020, 2021, 2022]
interval = "15min"
all_data = []

total_rows=0
for year in years:
    path=
    f"../../../../../../../../results/{year}/{floor_id}/{interval}/zone_stac_stanja_{f
loor_id}_{year}.csv"

    if Path(path).exists():
        df = pd.read_csv(path)

        filtered_df = df[ (df['speed'] == 33)]

        all_data.append(filtered_df)
    else:
        print(f"File not found: {path}")
if all_data:
```

```
combined_data = pd.concat(all_data, ignore_index=True)

combined_data.to_csv("../../../../../results/usporedbaThermal/24/speed1/15min/zones.csv", index=False)
```

Ponovno ponavljamo isti postupak , no uzimamo u obzir da nam za kalorimetar 24 trebaju fcu_type_id 1 i 2.

```
import pandas as pd

calorim= pd.read_csv("../../../../../results/calorimeter_filtered24.csv")

zones=
pd.read_csv("../../../../../results/usporedbaThermal/24/speed1/15min/zones.csv")

fcu_files=
pd.read_csv("../../../../../results/24/fcu_type_id1_with_zone.csv")

fcu_files1=
pd.read_csv("../../../../../results/24/fcu_type_id2_with_zone.csv")

koef = pd.read_csv("../../../../../data/fcu_hydraulic_model.csv")

all_fcu_files = pd.concat([fcu_files, fcu_files1], ignore_index=True)

calorim['timestamp']=
pd.to_datetime(calorim['timestamp']).dt.round('min')

zones['timestamp'] = pd.to_datetime(zones['timestamp']).dt.round('min')

all_fcu_files['timestamp']=
pd.to_datetime(all_fcu_files['timestamp']).dt.floor('min')

koef['timestamp'] = pd.to_datetime(koef['timestamp']).dt.round('min')

calorim = calorim.sort_values('timestamp')

zones = zones.sort_values('timestamp')

merged = pd.merge(zones, calorim, on='timestamp', how='left')
```

```
final_merged1 = pd.merge(merged, all_fcu_files, on=['timestamp',
'zone_id'], how='left')
```

```
koef = koef.rename(columns={'timestamp': 'timestamp_koef'})
```

```
final_merged = pd.merge(final_merged1, koef, on='fcu_id', how='left')
```

```
final_merged = final_merged.sort_values('timestamp')
```

```
final_merged.to_csv("../..../results/usporedbaThermal/24/speed1/15
min/merged_data.csv", index=False)
```

Iskoristili smo istu formulu za računanje modela kao i pri stvaranju modela.

Zatim ponovo pomoću linearne regresije i 3-sigma pravila filtriramo podatke, no ovaj puta ne mičemo podatke pri kojima je mass_volume_flow oko 0.

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.linear_model import LinearRegression
```

```
import numpy as np
```

```
zone_ids = [195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206,
207]
```

```
filtered_all = []
```

```
for zid in zone_ids:
```

```
    df_zone = df[(df['zone_id'] == zid) & (df['supply_temperature'] >=
10)].reset_index(drop=True)
```

```
    if df_zone.empty:
```

```
        print(f No data for zone_id {zid}, skipping...")
```

```
        continue
```

```
    X = df_zone[['supply_temperature']].values
```



```

y = df_zone['calculated_thermal_power'].values

model = LinearRegression()

model.fit(X, y)

y_pred = model.predict(X)

residuals = y - y_pred

sigma = np.std(residuals)

mask = np.abs(residuals) <= 3 * sigma

df_filtered = df_zone[mask].copy()

df_filtered['y_pred'] = y_pred[mask]

print(f"Zone {zid}: kept {df_filtered.shape[0]} / {df_zone.shape[0]}
points")

filtered_all.append(df_filtered)

final_filtered = pd.concat(filtered_all, ignore_index=True)

final_filtered.to_csv("../..../results/usporedbaThermal/24/speed1/
15min/merged_filtered.csv", index=False)

```

7.2. Sumiranje po svim zonama

Zatim smo tražili vremena u kojima imamo podatke za sve zone_id-jeve kako bi mogli usporediti sumu njihovih `calculated_thermal_power` sa `thermal_power` koji se očitava sa kalorimetra.

```

import pandas as pd

zone_ids = [195, 196, 197, 198, 199, 201, 202, 204, 205, 206, 207, 200,
203]

min_zones_required = 13

```

```

df_filtered = df[df['zone_id'].isin(zone_ids)].copy()
df_filtered['timestamp'] = pd.to_datetime(df_filtered['timestamp'])
df_filtered['minute'] = df_filtered['timestamp'].dt.floor('min')
df_unique = (
    df_filtered
    .drop_duplicates(subset=['minute', 'zone_id'])
)
grouped = df_unique.groupby('minute')

complete_minutes = grouped.filter(lambda g: len(g['zone_id'].unique()) >=
min_zones_required)

result = (
    complete_minutes.groupby('minute')['calculated_thermal_power']
    .sum()
    .reset_index()
    .rename(columns={'calculated_thermal_power': 'total_thermal',
'minute': 'timestamp'})
)
file_path_calorim = "../../../results/calorimeter_filtered24.csv"

```

7.3. Usporedba sa thermal_power na kalorimetru

```

file_path_calorim = "../../../results/calorimeter_filtered24.csv"

data_calorim = pd.read_csv(file_path_calorim)
data_calorim['timestamp'] = pd.to_datetime(data_calorim['timestamp'])
data_calorim['minute'] = data_calorim['timestamp'].dt.floor('min')
data_calorim.columns = data_calorim.columns.str.strip()

comparison_df = pd.merge(

```

```

    result,

    data_calorim[['minute', 'thermal_power']],

    left_on='timestamp',

    right_on='minute',

    how='left'

)

comparison_df['difference']      =      comparison_df['total_thermal']      -
comparison_df['thermal_power']

comparison_df['abs_difference'] = comparison_df['difference'].abs()


average_abs_difference = comparison_df['abs_difference'].mean()
max_abs_difference = comparison_df['abs_difference'].max()
min_thermal = comparison_df['total_thermal'].min()
mae = comparison_df['abs_difference'].mean()


comparison_df['percentage_error'] = (

    comparison_df['abs_difference']/
    comparison_df['thermal_power'].replace(0, np.nan)

)

mape = comparison_df['percentage_error'].mean(skipna=True) * 100

plt.figure(figsize=(12, 6))

plt.plot(comparison_df['minute'], comparison_df['total_thermal'],

         color='blue', label='Total Thermal (Calculated)')

plt.plot(comparison_df['minute'], comparison_df['thermal_power'],

         color='red', label='Thermal Power (Calorimeter)')

plt.xlabel("Timestamp (minute)")

plt.ylabel("Thermal Power (kW)")

plt.title("Line Comparison of Total Thermal vs Calorimeter Thermal Power")

plt.legend()

```

```

plt.grid(True)

plt.tight_layout()

plt.show()

# --- 3. Histogram ---

plt.figure(figsize=(8, 6))

plt.hist(comparison_df['difference'],          bins=30,          color='gray',
         edgecolor='black')

plt.xlabel("Difference (kW)")

plt.ylabel("Frequency")

plt.title("Distribution of Differences (Total - Calorimeter)")

plt.grid(True)

plt.tight_layout()

plt.show()

```

8. Rezultati

	R2	MSE (kW ²)	RMSE (kw)	MAE (kw)	MAPE (kw)	Max_err (kw)
jug(24)/fcu_type_1/speed0/15min	0.7589	78.2486	8.8454	6.7199	39.75	42.7261
jug(24)/fcu_type_1/speed0/30min	0.7870	75.1381	8.6677	6.5263	39.51	41.3392
jug(24)/fcu_type_1/speed0/1h	0.8028	72.0488	8.4875	6.3387	40.00	42.4839
jug(24)/fcu_type_1/speed1/15min	0.5015	109.7526	10.4752	8.2644	44.66	42.4100
jug(24)/fcu_type_1/speed1/30min	0.5342	101.0035	10.0480	7.8429	43.08	40.3501
jug(24)/fcu_type_1/speed1/1h	0.6423	94.9649	9.7366	7.4402	48.29	37.5443
jug(24)/fcu_type_1/speed2/15min	0.3724	107.6143	10.3704	8.2215	33.41	41.3814
jug(24)/fcu_type_1/speed2/30min	0.3914	94.2978	9.7094	7.7005	31.60	42.6348
jug(24)/fcu_type_1/speed2/1h	0.4380	80.5620	8.9688	7.0132	30.52	44.7689
jug(24)/fcu_type_1/speed3/15min	0.4861	115.7514	10.7506	8.3074	38.65	41.6838

jug(24)/fcu_type_1/speed3/30min	0.5132	102.1483	10.1003	7.9539	35.76	38.1861
jug(24)/fcu_type_1/speed3/1h	0.5840	81.9344	9.0306	7.1457	27.66	27.3264

Sl. 8.1 rezultati točnosti modela za kalorimetar 24 i fcu ID 1

	R2	MSE (kW ²)	RMSE (kW)	MAE (kW)	MAPE (kW)	Max_err (kW)
jug(24)/fcu_type_2/speed0/15min	0.8779	17.4713	4.1773	2.7990	22.23	27.9270
jug(24)/fcu_type_2/speed0/30min	0.7870	75.1381	8.6677	6.5263	39.51	41.3392
jug(24)/fcu_type_2/speed0/1h	0.8028	72.0488	8.4875	6.3387	40.00	42.4839
jug(24)/fcu_type_2/speed1/15min	0.6477	26.9355	5.1838	3.7980	30.56	27.0156
jug(24)/fcu_type_2/speed1/30min	0.6562	53.8923	7.5456	4.5645	35.56	36.4554
jug(24)/fcu_type_2/speed1/1h	0.6736	86.0726	9.2765	7.0236	44.77	44.8542
jug(24)/fcu_type_2/speed2/15min	0.5975	19.8535	4.4519	3.3331	18.09	20.6140
jug(24)/fcu_type_2/speed2/30min	0.6122	18.7584	4.2954	3.1539	18.04	20.3243

jug(24)/fcu_type_2/speed2/1h	0.6407	16.9336	4.1063	2.7767	16.02	23.7678
jug(24)/fcu_type_2/speed3/15min	0.5115	32.9661	5.5817	3.9251	25.99	24.0587
jug(24)/fcu_type_2/speed3/30min	0.5132	102.1483	10.1	7.9539	35.76	38.1861
jug(24)/fcu_type_2/speed3/1h	0.5027	93.2596	9.6397	7.6700	30.52	28.3317

Sl. 8.2 rezultati točnosti modela za kalorimetar 24 i fcu ID 2

	R2	MSE (kW ²)	RMSE (kW)	MAE (kW)	MAPE (kW)	Max_err (kW)
sjever(23)/fcu_type_2/speed0/15min/	0.8349	25.2889	5.0286	3.7130	25.94	30.7022
sjever(23)/fcu_type_2/speed0/30min/	0.8769	22.9182	4.7864	3.4050	27.20	33.5571
sjever(23)/fcu_type_2/speed0/1h/	0.8967	19.9503	4.4662	3.1785	26.49	30.7952
sjever(23)/fcu_type_2/speed1/15min/	0.7402	39.4093	6.2766	4.8190	25.38	33.5968
sjever(23)/fcu_type_2/speed1/30min/	0.7304	33.5174	5.7887	4.4222	21.69	24.7846
sjever(23)/fcu_type_2/speed1/1h/	0.7434	34.2031	5.8341	4.2644	27.04	38.2996
sjever(23)/fcu_type_2/speed2/15min/	0.6200	38.9940	6.2415	4.8501	19.45	27.3967
sjever(23)/fcu_type_2/speed2/30min/	0.6512	35.4466	5.9523	4.6270	19.22	29.3016

sjever(23)/fcu_type_2/speed2/1h/	0.6798	31.5962	5.6083	4.1379	21.58	27.2742
sjever(23)/fcu_type_2/speed3/15min/	0.7999	22.2488	4.7166	3.6718	14.64	28.7174
sjever(23)/fcu_type_2/speed3/30min/	0.8259	19.2418	4.3864	3.4166	14.39	26.9750
sjever(23)/fcu_type_2/speed3/1h/	0.9285	8.2935	2.8779	2.1756	12.30	14.6041

Sl. 8.3 rezultati točnosti modela za kalorimetar 23 i fcu ID 2

	MAE (kW)	MAPE (kW)
23/speed0/15min	1.1253884928254416	36.8055291609863
23/speed0/30min	1.1156002910737688	38.86633335711323
23/speed0/1h	1.1547936347813739	41.30946178988298
23/speed1/15min	2.337116261808347	37.05859522787472
23/speed1/130min	1.8719123104910986	33.5063824621807
23/speed1/1h	1.3027131166329555	28.25533957610667
23/speed2/15min	3.1806790177790534	35.403189484091705
23/speed2/30min	2.9523388871728935	33.84085467115304

23/speed2/1h	2.1327667042598275	28.29041976772404
23/speed3/15min	5.106909533037285	27.499594520114506
23/speed3/30min	4.769861850862218	25.443498881591616
23/speed3/1h	2.363790647897342	4.207395229514814

Sl. 8.4 usporedba toplinske energije za kalorimetar 23

	MAE (kw)	MAPE (kW)
24/speed0/15min	1.1970327095059674	28.907050097796954
24/speed0/30min	1.1970327095059674	28.907050097796954
24/speed0/1h	1.205170340961118	28.436925635882808
24/speed1/15min	1.507278875014867	16.611346740658576

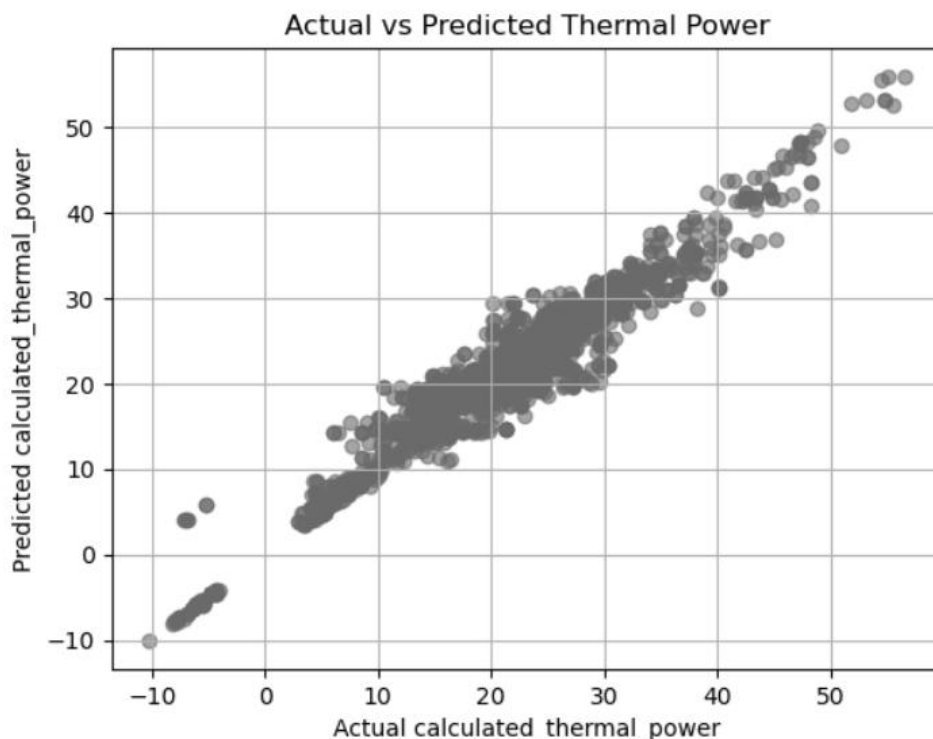
24/speed1/30min	1.1336819891995633	11.557281988128668
24/speed1/1h	0.8032250990999883	7.692913030231037
24/speed2/15min	2.3768032546715623	29.118801296624774
24/speed2/30min	2.255895191736245	34.233401185917685
24/speed2/1h	1.4779118889288498	3.0205401025452323
24/speed3/15min	1.9552645392	10.032063764379256
24/speed3/30min	1.736374410113107	8.51705544748694
24/speed3/1h	1.6710749455959484	3.5402456920148864

Sl. 8.5 usporedba toplinske energije za kalorimetar 24

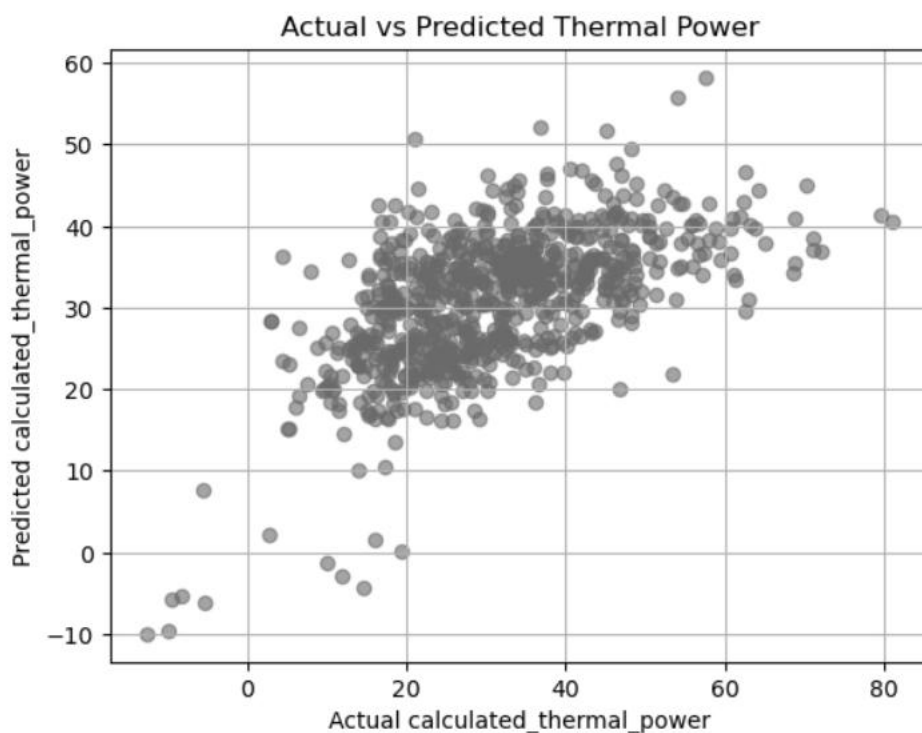
9. Zaključak

9.1. Model

Kao što možemo uočiti iz rezultata točnost modela se povećava sa intervalom. Uzrok tome je što uzimanjem većeg intervala za stabilno stanje eliminiramo slučajeve u kojima je stanje u svim zonama bilo isto kraći vremenski period. Najveće točnosti dobivamo pri speed0 zato što pri kreiranju tih modela imamo najviše podataka. No ti podatci nam nisu bitni jer je nama bitna predikcija rada ventilokonvektora dok je on upaljen. Jedino se anomalija događa kod skupa podataka za model jug(24)/fcu_type_2/speed3 pri čemu se povećanjem intervala točnost smanjuje. Razlog tome je što model ima vrlo malu količinu podataka s kojima bi se trenirao. U tome slučaju gubimo benefit filtriranja podataka na veći interval kako bi povećali točnost. Najveću točnost postiže sjever(23)/fcu_type_2/speed3/1h sa R2 vrijednosti 0.9285, dok najmanju točnost ostvaruje jug(24)/fcu_type_1/speed2/15min sa 0.3724.



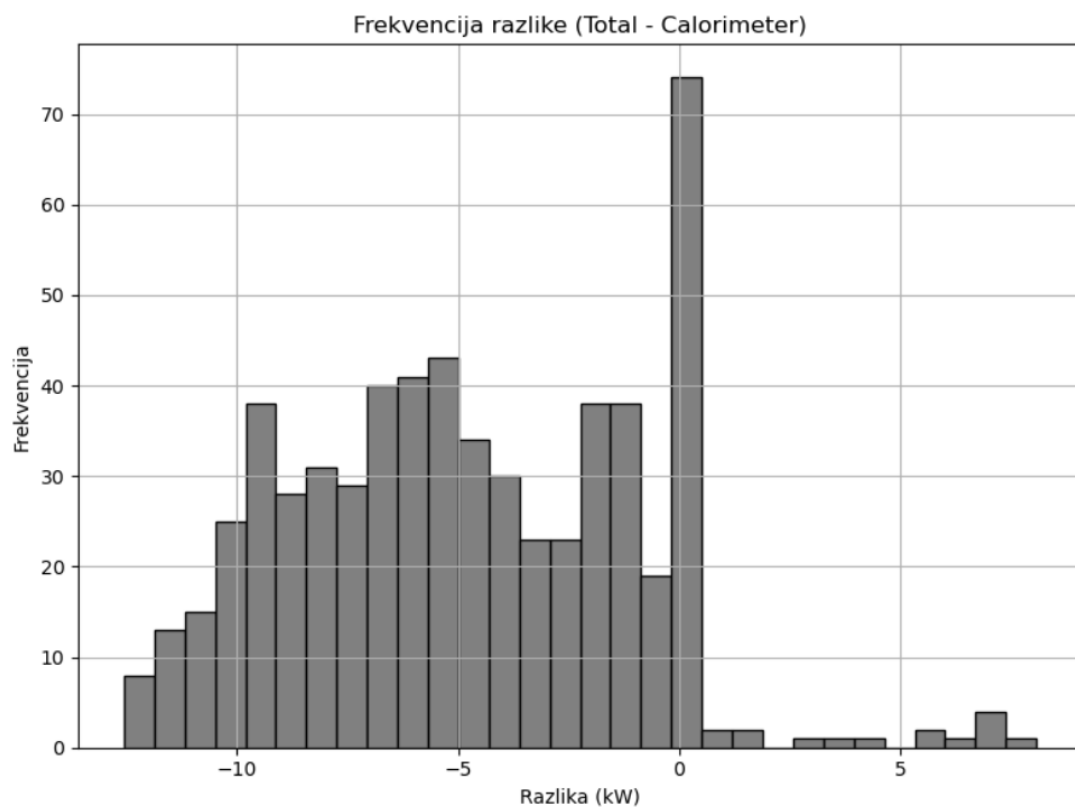
Sl. 9.1 graf modela za sjever(23)/fcu_type_2/speed3/1h



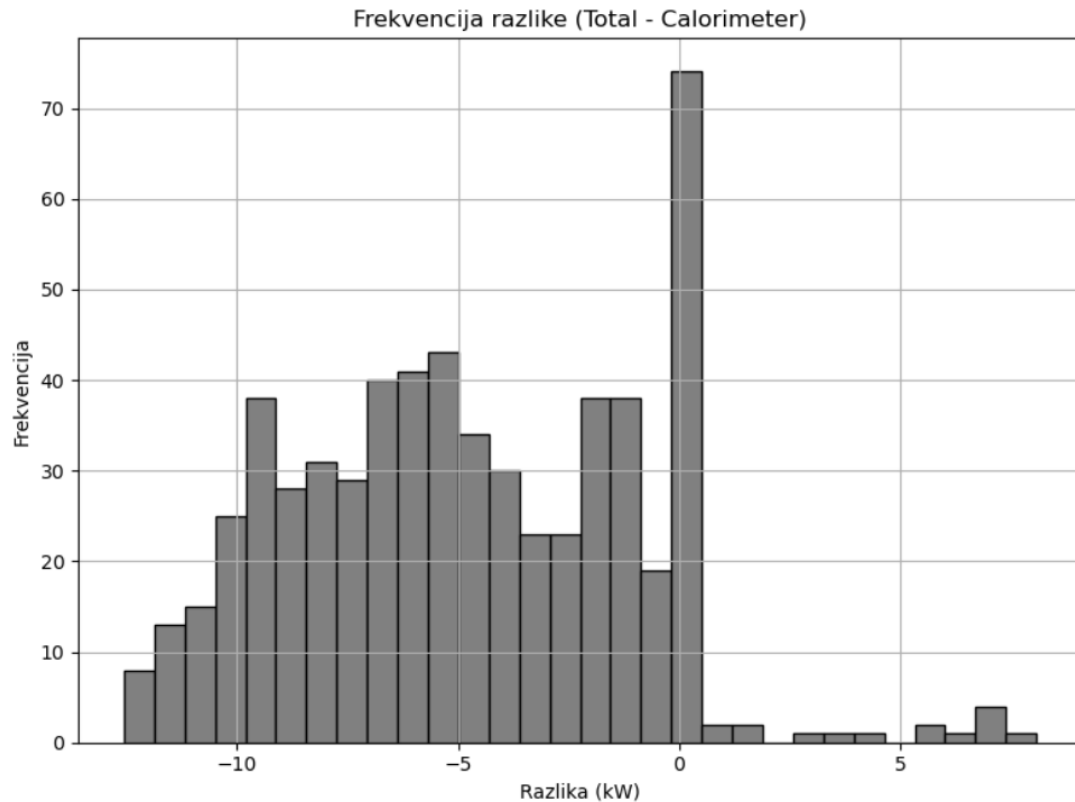
Sl. 9.2 graf modela za jug(24)/fcu_type_1/speed2/15min

9.2. Usporedba toplinske snage

Kod uspoređivanja izračunatog `thermal_power` sa usporedbom `thermal_power` očitano na kalorimetru dolazimo do sljedećih rezultata. Za `speed 0` u svim intervalima dobivamo sličnu pogrešku zbog toga što za te slučajeve imamo veliku količinu podataka pa veličina intervala ne radi toliku razliku. Za ostale slučajeve možemo uočiti da greška (MSE) pada što je interval veći. Najveći MSE postižemo za `23/speed3/15min` koji iznosi `5.106909533037285` dok se najmanji MSE postiže za `24/speed1/1h` i iznosi `0.8032250990999883`. Razlog tome je što za `23/speed3/15min` imamo malu količinu podataka budući da je brzina 3 među rjeđim pojavama, dok je brzina 1 već češća pojava. Tome također pridonose i intervali. Što je odabrani interval dulji točnost bi trebala biti veća.



Sl. 9.3 usporedba toplinske snage za za 23/speed3/15min



Sl. 9.4 usporedba toplinske snage za 24/speed1/1h

Ac
Go

Ac
Go

10. Sažetak

Cilj projekta je bio napraviti modele pomoću strojnog učenja kojim se predviđa potrošnja termalne energije u sustavi. Time se pokušava optimizirati rad ventilokonvektora. Kako bi to realizirali trebalo je ispuniti dvije ključne stvari. Prvi je bio napraviti model pomoću strojnog učenja (black-box princip) kako bi mogli predvidjeti kojom brzinom ventilokonvektor mora djelovati za prisutnu temperaturu zone. Drugi zadatak je bio napraviti sumirati izračunate termalne snage sa svih zona pridijeljenih određenom kalorimetru sa vrijednosti termalne snage očitane sa kalorimetra. Iako u rezultatima spominjemo brzinu 0 najbitniji su nam podatci pri brzinama 1, 2 i 3 jer smo usredotočeni na učenje rada stroja dok je on upaljen, no pri brzini 1 je on ugašen. Najpreciznije modele smo postigli za stacionarne intervale od 1h kao što smo i očekivali.

Kod usporedbe sumirane izračunate termalne snage sa svih zona pridijeljenih određenom kalorimetru sa vrijednosti termalne snage očitane sa kalorimetra smo dobili različite točnosti pri različitim brzinama iz razloga što je model učen sa drugačijom količinom podataka (sa više podataka se postiže veća točnost).

Predložena su poboljšanja za budući rad, uključujući nove podatke i nove metode strojnog učenja kako bi mogli poboljšati točnost naučenih modela. Također bi pri optimiranju modela od velike važnosti bilo detaljnije shvaćanje cijelog sustava i raznih pojava koje mogu uzrokovati kriva mjerenja.

Ključne riječi: rainforest, black-box, strojno učenje, ventilokonvektor, FCU, linearna regresija

11. Literatura

[1.] Learning Model Building in Scikit-learn (ožujak, 2025.), Poveznica: <https://scikit-learn.org/1.4/tutorial/index.html>

[2.] An introduction to machine learning with scikit-learn (ožujak, 2025.) , Poveznica: <https://www.geeksforgeeks.org/machine-learning/learning-model-building-scikit-learn-python-machine-learning-library/>

.